CSCE 5610 Computer Architecture

Comparative Analysis of FIFO and LRU Page Replacement Algorithms in Virtual Memory

Group 4

Team Members

Ruchitha Sri Sai Valli Thanmayee Potladurthy – 11855553

Adla Sanjay Ramaswamy – 11828288

Dharani Kondapally – 11853256

GitHub Repository:

https://github.com/Dharanikondapally/Group04_CSCE5610_FA26

1. Objective

We will create a virtual memory simulator in Python that compares FIFO and LRU. Both will process the same page-reference string in different memory-frame configurations, measuring the number of page hits, misses, and replacements.

How FIFO and LRU differ in terms of page hits and misses, when processing the same task in different frame configurations?

2. Architecture Concept

A page fault occurs when the reference page is not present in memory and has to be replaced, if all frames are busy. FIFO exchange the longest-hosted page, no matter its last access time, LRU will swap out the least recently used page ev. The changes will be evaluated in the same working conditions.

3. Design and Artifact

The modular Python simulator will consist of the experiment controller, FIFO and LRU implementations, metric collection, and comparison of results.

It takes page-reference string and input number of frames, thus applying same inputs for both algorithms in a sequential manner with independent frame statuses.

- FIFO: Keeps track of insertions and swaps oldest page out.

- LRU: Tracks usage of pages and swaps out least recently used page.

System Workflow

The controller of the experiment gives equal page-reference sequences and number of frames to both FIFO and LRU for autonomous calculations. The results are then gathered and compared according to standard parameters such as step-by-step frame states, total references, hits, faults, fault rate, successful rate, and finally frame state.

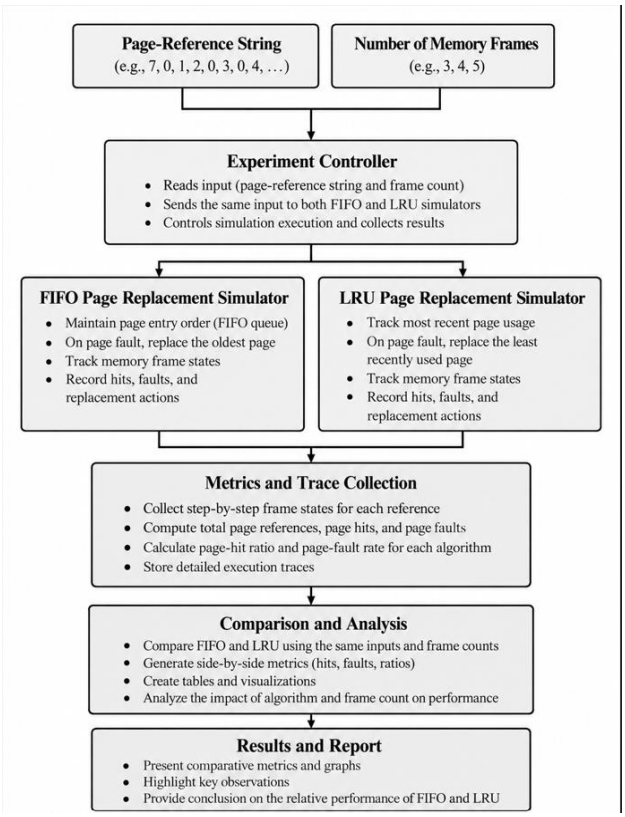


Figure 1. Workflow of the FIFO and LRU Virtual Memory Simulator.

4. Comparison

Both procedures will be assessed in accordance with the same situation:

- FIFO: a page that was loaded first will always be substituted when a fault is encountered and every frame is occupied

- LRU: it is going to remove the page that was used the least recently in these conditions

It will be noted that identical strings of reference will be used in both algorithms. Various numbers of frames will be chosen (3, 4, and 5 in the beginning) so that the difference in the results will be attributed to the methods of replacement used but not to the reference string itself.

5. Metric and Test Plan

The key measurements include page faults, fault fraction, page hit count and hit ratio:

Page-Fault Rate = Page Faults / Total References Page-Hit Ratio = Page Hits / Total References

FIFO and LRU will be examined using the multiple given reference sequences and frame counts. Opportunities for evaluation are possible since both algorithms are deterministic, resulting in workloads being varied rather than repeating many times.

Correctness verification will rely on analyses of manually traceable cases which will allow the comparison of frames' states and replacement decisions, hits and faults against the simulator outputs and will allow checking for the following parameters:

- Page Hits + Page Faults = Total References

- Resident Pages ≤ Available Frames

6. Tools and Dependencies

Component	**Tool / Environment**
Programming Language	Python 3
Core Libraries	Python standard library
Development Environment	VS Code or another Python-capable environment
Version Control	Git
Repository	GitHub
Project Management	GitHub Issues and GitHub Projects
Hardware	Standard computer capable of running Python 3
Starter Code	None required
External Simulator	None required

The independent simulator operates without additional hardware, special rights for operating systems or external data. The file input already contains its contents so there is nothing new in experiments.

7. Schedule and Roles

Timeline	**Major Tasks**	**Owner(s)**
Week 1 – Planning and Design	Establish the required structure of the simulator, format of inputs and outputs, list of workloads and frame, performance evaluation criteria and tests to ascertain that simulation is carried out in compliance with established norms.	All Members
Week 2 – Core Implementation	Create FIFO and LRU and integrate the capabilities of the simulator to detect page hits or faults as well as logic of replacement and keeping records of the condition of each frame.	Dharani – FIFO; Sanjay – LRU

Week 3 – Integration and Checkpoint	Combine the two algorithms with a single experiment controller, analyze performed operations, and check the program against simplest test scenarios.	Ruchitha – Integration; All Members – Testing
Week 4 – Testing and Experimentation	Conduct experiments and analyze the findings, for instance, by comparing the results of FIFO and LRU operations and putting real-life data in scientific format in terms of charts and tables.	All Members
Week 5 – Analysis and Results	Compare FIFO and LRU results, organize measurements into tables and visualizations, and analyze differences across workloads and frame configurations.	Ruchitha & Sanjay – Analysis; Dharani – Validation
Week 6 – Finalization	Perform final testing, clean and document the code, finalize repository documentation, organize final results, and prepare the report and presentation materials.	All Members

The team is expected to have a functioning version of the project that can process similar inputs using FIFO and LRU algorithms and produce validated frame traces, page-hit counts, page-fault counts, hit ratios, and fault rates by the end of Week 3. The remaining weeks will be focused on controlled experimentation, analysis, documentation, and final delivery.

8. Risks and Backup Plan

Risk 1: Errors in the page replacement strategy or in the metric measurements: Mistakes in replacing pages, placing orders for replacement pages, or tracking of the most recent uses may lead to wrong outcomes.

Mitigation/Backups: To handle this risk, the following procedures will be carried out. Reference chains that are short and easy to follow will be used for validating the algorithms. These chains will be used to connect the expected number of frame transitions with the simulator's output. Since FIFO and LRU algorithms will be executed as two independent modules, they can be checked out independently in case there is a problem with the integration.

Risk 2: Difficulties with visualization or delayed analysis: Problems arising in plotting and presentation will admit extra time spent on validating the simulator.

Mitigation and Backup: The basic implementation, accuracy testing, and experimental information gathering must be accomplished prior to the optional visualization phase. In the event that different visualizations cannot be performed satisfactorily, validated comparison tables and trace frames may be used to convey the results. This allows for the essential FIFO-LRU comparison to be made in the scheduled time of the project.